

MINI PROJECT REPORT
ON
REPOSYS -- CAMPUS REPROGRAPHY AUTOMATION SYSTEM

Submitted By

MGP22NMC034	JEFRI JIJI
MGP22NMC037	JERIN SEBASTIAN
MGP22NMC050	SETHULAKSHMI P.S

To

*the APJ Abdul Kalam Technological University in partial fulfillment of the
requirements for the award of the degree of*

Master of Computer Applications

Under the Guidance of

Dr. Abin T. Abraham

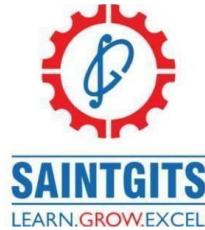


DEPARTMENT OF COMPUTER APPLICATIONS
Saintgits College of Engineering (Autonomous) Pathamuttom,
Kottayam, Kerala-686532

NOVEMBER 2025

SAINTGITS COLLEGE OF ENGINEERING (AUTONOMOUS)

Kottukulam Hills, Pathamuttom, Kottayam, Kerala



BONAFIDE CERTIFICATE

Certified that the report entitled **“REPOSYS -- Campus Reprography Automation System”** submitted by **Jefri Jiji MGP22NMC034, Jerin Sebastian MGP22NMC037** and **Sethulakshmi P.S MGP22NMC050** to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the Degree of **Master of Computer Applications** is a bonafide record of the project work carried out by them under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Dr. Abin T. Abraham

Internal Guide

Dr. Rani Saritha R

Project Co-ordinator

Dr. Rani Saritha R

Head of the Department

Dr. Sudha T

Principal

Viva-voce held on:

ACKNOWLEDGEMENT

At the outset, we thank the lord almighty for his abundant grace, strength and hope to make our endeavour a success. We express our deep-felt gratitude to **Dr. Sudha T.**, Principal, Saintgits College of Engineering (Autonomous) for her warm support with regard to the work and to the management for providing the facilities required.

We would like to place our deep sense of gratitude to **Prof. Mini Punnoose**, Director- MCA, Department of Computer Applications, Saintgits College of Engineering (Autonomous) for her constant encouragement. We express our gratitude to **Dr. Rani Saritha R.**, Head, Department of Computer Applications, Saintgits College of Engineering (Autonomous), for her support and guidance.

We profoundly grateful to **Dr. Abin T. Abraham**, Assistant Professor, our project guide and **Dr. Rani Saritha R.**, the project co-ordinator for their valuable guidance, help, suggestions and assessment.

Furthermore, we would like to thank all others especially our parents and numerous friends. This project report would not have been a success without their inspiration, valuable suggestions and moral support from them throughout its course.

Jefri Jiji
Jerin Sebastian
Sethulakshmi P.S

ABSTRACT

In educational institutions, hospitals, corporate offices, and other secure environments, the conventional manual document printing and reprography counter system poses significant challenges including manual billing errors, lack of verification, time-consuming physical queues, and inadequate security measures. The web-based and IoT-integrated Reprography Automation System (REPOSYS) addresses these critical issues by introducing a fully automated, secure, and efficient digital solution that transforms campus reprography management through modern web and systems technology.

The system operates through an intuitive responsive web portal and tablet kiosk interface. As soon as a student, faculty member, or visitor accesses the portal or counter kiosk, a file processing engine powered by pdf-lib and client-side web technologies automatically inspects their documents, extracting page counts, identifying blank pages, and detecting color modes. If the user has previously registered, their profile details and digital wallet balance are automatically retrieved and pre-filled in the order form, significantly reducing submission time and enhancing user convenience. For first-time visitors or guests, the system prompts them to complete a quick registration or creates a time-bound guest kiosk session. In all cases, users specify their exact printing requirements including copies, color mode, duplexing, and binding. To ensure authenticity, a 4-digit One-Time Password (OTP) and QR code token are generated for physical pickup verification at the counter.

The software architecture is built using Node.js and the Express web framework for backend operations, handling routing, session management, and API endpoints for order lifecycle management, pricing, and OTP verification. The frontend interface uses React 19, HTML, CSS, and Tailwind CSS for a responsive, modern user experience. Socket.IO enables real-time queue synchronization, live wait-time updates, and instant status transitions across connected client and staff dashboards. For online payment processing, the system integrates the Razorpay payment gateway API alongside an internal ACID-compliant digital wallet. All operational data including user credentials, order records, timestamps, payment transactions, and inventory items are securely stored in a cloud MongoDB Atlas database.

A key feature is the staff and administrator dashboard, which provides authorized personnel with complete control and visibility over printing queues and operational records. Operators can view real-time queue entries, verify cash collections, trigger physical print jobs via a native Electron Print Agent, search and filter logs by date or

service type, and export daily reports for auditing. The dashboard displays active jobs alongside customer details, enabling quick verification and eliminating manual paper registers.

The system enhances security by addressing multiple vulnerabilities in traditional reprography management. Digital file transfer eliminates the need for uninspected USB drives and unencrypted mobile messaging, establishing a secure digital audit trail. Real-time OTP verification prevents wrong-order handovers and unauthorized collections. The automated pricing engine eliminates manual calculation mistakes and exact change shortages. The Composite Priority Queue algorithm with dynamic aging deduction guarantees fairness for students while prioritizing urgent faculty requisitions, eliminating queue starvation.

Beyond educational campuses, the Reprography Automation System has broad applicability across corporate document centres, commercial print shops, legal chambers, government secretariats, and co-working spaces. The relevance of this project lies in its alignment with the growing emphasis on digital transformation, paperless automation, and operational accountability in public and semi-public facilities. By replacing outdated manual registers and physical counter bottlenecks with an intelligent, automated platform powered by MERN, Socket.IO, Razorpay, and Electron, the system improves operational throughput while significantly enhancing security, transparency, and resource sustainability.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION	1
1.1 Introduction to the Project	1
1.2 Organization Profile	1
1.3 Objectives of the Project	2
1.4 Scope and Applicability	3
CHAPTER 2: REQUIREMENTS AND ANALYSIS	4
2.1 Existing Systems	4
2.2 Proposed System	4
2.3 Feasibility Study	5
2.3.1 Technical Feasibility	6
2.3.2 Operational Feasibility	6
2.3.3 Economic Feasibility	6
2.3.4 Time Feasibility	6
2.4 Conceptual Modelling	7
2.5 Planning and Scheduling	8
CHAPTER 3: SYSTEM SPECIFICATION	10
3.1 Software and Hardware Requirements	10
3.1.1 Software Requirements	10
3.1.2 Hardware Requirements	11
3.2 Functional Specifications	11
3.3 Tools and Platforms Used	12
CHAPTER 4: SYSTEM DESIGN	15
4.1 Module Description	15
4.1.1 User Authentication Module	15
4.1.2 Document Upload and Analysis Module	15
4.1.3 Order Configuration Wizard Module	16
4.1.4 Dynamic Cost Estimation Module	16
4.1.5 Payment and Digital Wallet Module	16
4.1.6 Composite Priority Queue Module	17

4.1.7 Counter Staff Operations Module	17
4.1.8 OTP Pickup Handover Module	17
4.1.9 Administrator Governance Module	18
4.1.10 Consumable Inventory Tracker Module	18
4.1.11 Desktop Print Agent Module	18
4.2 Schema Design	19
4.2.1 Database Tables	19
4.2.2 Relationships	21
4.2.3 Constraints	21
4.3 Procedural/Flow Design	21
4.3.1 Order and Production Flowchart	21
4.4 User Interface Design	23
4.4.1 Client Interface	24
4.4.2 Administrator and Staff Dashboard	24
CHAPTER 5: AGILE METHODOLOGY	26
5.1 Project Roadmap	26
5.2 User Stories	27
5.2.1 User Stories	27
5.2.2 Sprint Planning	28
5.3 Test Plan	29
CHAPTER 6: IMPLEMENTATION AND TESTING	31
6.1 Implementation Procedures	31
6.2 Testing Methods and Results	32
6.2.1 Unit Testing	32
6.2.2 Integration Testing	33
6.2.3 System Testing	33
6.2.4 User Acceptance Testing	33
CHAPTER 7: CONCLUSIONS	35
7.1 Summary	35
7.2 Limitations	35
7.3 Future Scope	36
CHAPTER 8: APPENDICES	37

8.1 Code	37
8.2 Screenshots	65
CHAPTER 9: REFERENCES	71

LIST OF TABLES

Table 4.1 Users Table	19
Table 4.2 Orders Table	20
Table 4.3 Documents Table	20
Table 4.4 Payments Table	20
Table 5.1 Project Roadmap	26
Table 5.2 Sprint Planning Table	28

LIST OF FIGURES

Figure 2.1 ER-Diagram of Proposed System	8
Figure 3.1 Use Case Diagram	12
Figure 4.1 Order Placement and Production Flowchart	23

CHAPTER 1: INTRODUCTION

1.1 Introduction to the Project

The Reprography Automation System (REPOSYS) replaces manual campus print shop operations and paper logbooks with a secure, automated digital platform and counter kiosk. In contemporary academic institutions, document printing, copying, scanning, and binding are vital daily requirements for students submitting assignments, laboratory records, seminar reports, and theses, as well as faculty members preparing examination question papers and administrative course materials. Traditionally, students and faculty must physically walk to the campus reprography centre, stand in long lines, share sensitive documents over personal WhatsApp messaging or uninspected USB drives, wait for manual cost calculations, and pay in cash. This manual workflow creates severe counter bottlenecks during peak submission periods, exposes personal phone numbers, risks malware transmission, and leads to uncollected prints that generate substantial paper waste.

REPOSYS modernizes this environment by providing a unified web-based portal and counter kiosk. Students and faculty submit document duplication requests online, where an automated pre-flight file inspection engine extracts exact page counts and detects blank pages. Real-time cost computation applies transparent institutional tariffs, including paper conservation incentives. Secure payments are settled via Razorpay, an ACID-compliant student digital wallet, or Pay at Counter cash collections. At the counter, operators manage incoming production queues through a real-time Socket.IO dashboard, trigger automated printing to local Windows hardware via an Electron desktop Print Agent, and enforce One-Time Password (OTP) verification before releasing finished documents.

1.2 Organization Profile

Saintgits College of Engineering, established in 2002, is a premier private self-financing institution located in Pathamuttom, Kottayam, Kerala. Affiliated with APJ Abdul Kalam Technological University (KTU) and approved by the All India Council for Technical Education (AICTE), the college has earned recognition for its academic excellence, state-of-the-art infrastructure, and forward-looking research initiatives. The institution offers a diverse portfolio of undergraduate and postgraduate programs, including Integrated Master of Computer Applications (IMCA), MCA, M.Tech, and B.Tech disciplines. Saintgits holds autonomous status granted by the University Grants Commission (UGC), making it one of the first three engineering colleges in Kerala to

achieve this distinction, which enables it to design its own curriculum, conduct examinations, and implement innovative teaching methods to maintain high educational standards. With nine NBA-accredited programs and a campus equipped with advanced laboratories, research centers, and modern facilities, Saintgits fosters a dynamic and conducive learning environment. Beyond academics, the college promotes holistic development through technical fests, cultural events, and sports, while its strong industry collaborations and excellent placement record further establish its reputation as a leading engineering institution in Kerala.

1.3 Objectives of the Project

The primary objective of this project is to develop a cloud-connected Reprography Automation System that automates campus document printing workflows while enhancing security, fairness, and operational efficiency. Specific objectives include:

1. **Automation of Order Submission and Processing:** Replace manual USB file transfers and physical counter queues with a digital upload portal to reduce delays, save student time, and ensure accurate record-keeping.
2. **Enhanced Handover Security:** Implement One-Time Password (OTP) and QR-code verification at the counter to prevent unauthorized document collections and protect user confidentiality.
3. **Real-Time Queue Monitoring and Transparency:** Provide an Administrator and Staff Dashboard to track live queue positions, monitor print jobs, manage inventory, and generate exportable reports for administrative auditing.
4. **Dynamic Priority and Fairness:** Establish an anti-starvation priority queue engine combining academic role weights with waiting-time aging deductions to prevent job starvation during peak submission hours.
5. **Multi-Channel Financial Settlement:** Facilitate cashless student transactions via integrated Razorpay online payments, an ACID-compliant digital student wallet, and regulated Pay at Counter cash handling.
6. **Data Integrity and Traceability:** Ensure that all order parameters---including page counts, duplex settings, timestamps, costs, and audit logs---are securely stored and readily retrievable for future institutional reconciliation.

1.4 Scope and Applicability

The Reprography Automation System is designed to modernize campus reprography centres by automating registration, file analysis, cost calculation, payment, and handover verification processes. The system primarily targets educational institutions, such as college and university campuses, where large numbers of students, research

scholars, and faculty members require fast, dependable, and secure document duplication services. By replacing manual logbooks and physical line-standing with a web portal and counter terminal integrated with Razorpay payments, Socket.IO real-time updates, and an Electron desktop Print Agent, the system improves accuracy, reduces staff workload, and enhances overall campus operational efficiency.

Beyond tertiary educational institutions, the system is scalable and adaptable to other document-intensive environments, including corporate offices, hospitals, government secretariats, research facilities, law libraries, and commercial print establishments. Its modular architecture allows integration with additional hardware and software security measures, such as campus RFID smart cards, multi-printer load balancers, real-time audio alerts, and automated SMS/email notifications, making it suitable for a wide range of operational contexts.

CHAPTER 2: REQUIREMENTS AND ANALYSIS

2.1 Existing Systems

Currently, most educational institutions, including Saintgits College of Engineering, rely on manual procedures and physical logbooks to record and manage printing orders. Students and faculty are required to walk to the reprography centre, wait in physical queues, share files via uninspected USB flash drives or personal WhatsApp chats, calculate costs manually, and pay using physical cash. This traditional workflow has several critical limitations:

- **Time-consuming and Inefficient:** Manual file transfer and manual page counting slow down counter check-ins, especially during peak submission hours, causing extensive lobby congestion and lost study time.
- **Error-prone and Inaccurate:** Handwritten receipts and manual tariff calculations are susceptible to arithmetic mistakes, wrong duplex selections, and billing inaccuracies that cause revenue loss.
- **Limited Security and Privacy Exposure:** Sharing academic reports and identity documents over personal mobile messaging platforms exposes phone numbers and personal files to counter staff and third-party servers.
- **Lack of Real-Time Monitoring:** Students cannot check counter queue lengths or machine availability before visiting the facility, leading to unpredictable waiting times.
- **Poor Data Management and Wastage:** Physical paper logbooks are difficult to store, search, and audit. Furthermore, uncollected printouts lead to significant paper and toner waste that cannot be reclaimed.

2.2 Proposed System

The proposed Reprography Automation System aims to overcome the limitations of traditional print shop management methods by introducing a fully automated, secure, and intelligent digital platform. Instead of maintaining physical registers and accepting uninspected USB drives, the system provides a responsive web application and counter terminal integrated with cloud storage, real-time WebSockets, and a native desktop Print Agent.

When a student or faculty member uploads a document, the system automatically analyzes the file using pdf-lib, extracting page numbers and flagging blank pages. The customer customizes printing parameters (copies, color mode, duplexing, binding) and

receives an instant cost quotation computed from active institutional rates. Online payments are settled via Razorpay or an internal digital wallet backed by MongoDB ACID transactions, or marked as Pay at Counter.

All operational data---such as order details, user credentials, timestamps, payment records, and pickup OTPs---are securely stored in a cloud MongoDB database. The system also features an intuitive Staff and Administrator Dashboard that allows authorized counter operators to view active priority queues, verify cash payments, dispatch physical print jobs, and generate comprehensive daily reports for administrative auditing.

2.3 Feasibility Study

Before implementing the Reprography Automation System, a detailed feasibility study was conducted to ensure that the project is practical, cost-effective, and technically achievable.

2.3.1 Technical Feasibility

The proposed system is technically feasible since it utilizes readily available and proven technologies such as React 19, Node.js, Express.js, MongoDB, Socket.IO, and Electron. These components are lightweight, modern, open-source, and cross-platform. Integrating automated PDF pre-flight inspection and desktop printer spooling is readily achievable using pdf-lib and pdf-to-printer libraries. The use of cloud database clusters (MongoDB Atlas) and cloud storage (Cloudinary) ensures smooth data synchronization and high availability.

2.3.2 Operational Feasibility

From an operational perspective, the system is user-friendly and requires minimal training for campus stakeholders. Students and faculty can easily submit orders from smartphones or laptops, and counter staff can manage queues through an intuitive, streamlined dashboard. Automating document validation and OTP pickup verification simplifies counter operations, reducing human error.

2.3.3 Economic Feasibility

The project is economically feasible as it primarily relies on open-source frameworks, minimizing software licensing expenditures. Hardware requirements are limited to existing counter PCs, standard multi-function commercial printers, and campus Wi-Fi network infrastructure---all of which are already available on campus. In the long run, it saves operational costs by preventing abandoned prints, reducing paper waste, and eliminating billing discrepancies.

2.3.4 Time Feasibility

The system can be developed within a reasonable academic timeframe using modular, Agile development techniques. Each subsystem---authentication, document analysis, pricing engine, payment gateway, priority queue, and staff dashboard---can be developed and verified independently before final integration.

2.4 Conceptual Modelling

The conceptual model provides a high-level understanding of how data flows within the Reprography Automation System. It visually represents the relationships between different entities involved in reprography operations, assisting in the design of a normalized database schema.

Figure 2.1: ER-Diagram of Proposed System

2.5 Planning and Scheduling

The development of the Reprography Automation System was carried out in a structured and phased manner across 16 weekly sprints, adhering to the KTU project curriculum and Saintgits Scrum evaluation manual.

CHAPTER 3: SYSTEM SPECIFICATION

3.1 Software and Hardware Requirements

3.1.1 Software Requirements

Operating System: Windows 10/11 or Linux for backend server; Windows 10/11 64-bit for Counter Workstation; Android, iOS, Windows, macOS for client users.

Languages and Frameworks: Node.js v20.x LTS, Express.js v4.19, React 19, Tailwind CSS, Electron v30.x.

Libraries & Tools: Mongoose ODM, pdf-lib, Socket.IO v4.8, Razorpay SDK, pdf-to-printer, node-cron, Selenium WebDriver.

3.1.2 Hardware Requirements

Counter Workstation: Desktop PC with Intel Core i3 10th Gen, 8 GB RAM, 256 GB SSD, USB 3.0 / Gigabit LAN.

Production Server: Quad-Core CPU (2.4 GHz), 4 GB RAM, 20 GB SSD, 100 Mbps uplink.

Printers: Heavy-duty commercial multifunction printers (Canon, HP, Ricoh, Konica Minolta).

3.2 Functional Specifications

The key functional capabilities of REPOSYS include user authentication with role-based access control, pre-flight PDF analysis, custom order configuration, dynamic pricing with duplex paper discounts, multi-channel payment processing, composite priority queue with dynamic aging, OTP handover verification, automated desktop print spooling, and administrative governance.

3.3 Tools and Platforms Used

The platform leverages React 19 for reactive frontend rendering, Node.js and Express for backend REST APIs, MongoDB Atlas for ACID-compliant document storage, Socket.IO for real-time WebSocket distribution, Cloudinary for signed media hosting, Razorpay for cashless payments, and Electron for local Windows print spooling.

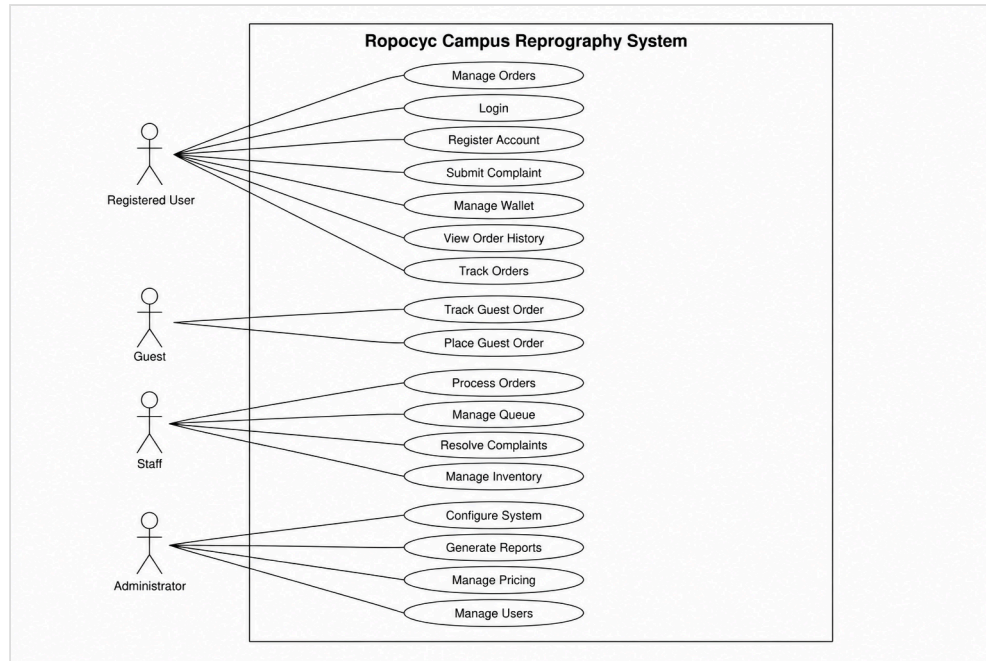


Figure 3.1: Use Case Diagram

CHAPTER 4: SYSTEM DESIGN

4.1 Module Description

The system comprises 11 functional modules: User Authentication, Document Upload and Analysis, Order Configuration Wizard, Dynamic Cost Estimation, Payment and Digital Wallet, Composite Priority Queue, Counter Staff Operations, OTP Pickup Handover, Administrator Governance, Consumable Inventory Tracker, and Desktop Print Agent.

4.2 Schema Design

4.2.1 Database Tables

Table 4.1: Users Table

Field	Data Type	Description
id	ObjectId	Unique identifier for each user (Primary Key)
name	String	User's full name
email	String	Institutional email address (unique)
password	String	Salted bcrypt password hash
role	String	Role classification (Student, Faculty, Staff, Admin)
department	String	Academic department
walletBalance	Number	Current pre-funded digital wallet balance
isVerified	Boolean	Email verification status flag

Table 4.2: Orders Table

Field	Data Type	Description
id	ObjectId	Unique identifier for each order (Primary Key)
orderNumber	String	Human-readable tracking identifier (e.g. ORD-10024)
userId	ObjectId	Foreign key reference to Users collection
serviceType	String	Service category (Printing, Copy, Scan, Binding)
totalCost	Number	Total price in INR
paymentStatus	String	Settlement status (Pending, Paid, Cash_Pending)
status	String	Order state (Pending, In_Queue, Processing, Ready, Completed)
priorityScore	Number	Composite score governing queue position
pickupOtp	String	4-digit secret code for handover verification

Table 4.3: Documents Table

Field	Data Type	Description
id	ObjectId	Unique identifier for document record (PK)
orderId	ObjectId	Foreign key linking to Orders collection
fileName	String	Original client file name
fileUrl	String	Secure signed cloud storage URL
pageCount	Number	Total verified pages in document
colorPages	Number	Count of pages containing color elements
fileSize	Number	Document size in bytes

Table 4.4: Payments Table

Field	Data Type	Description
id	ObjectId	Unique payment identifier (PK)
orderId	ObjectId	Foreign key reference to Orders collection
userId	ObjectId	Foreign key reference to Users collection
amount	Number	Amount settled in INR
method	String	Payment channel (Razorpay, Wallet, Cash)
transactionId	String	Gateway transaction reference ID
status	String	Settlement status (Completed, Failed, Pending)

4.2.2 Relationships

The system establishes one-to-many relationships between Users and Orders, Orders and Documents, and Users and Payments, with referential integrity enforced via Mongoose schema validators.

4.2.3 Constraints

Primary key uniqueness, required field validation, and domain-restricted email checks ensure strict database consistency.

4.3 Procedural/Flow Design

4.3.1 Order and Production Flowchart

Figure 4.1: Order Placement and Production Flowchart

4.4 User Interface Design

4.4.1 Client Interface

The client portal offers a mobile-first wizard stepper, instant price estimation card, and live queue tracking timeline.

4.4.2 Administrator and Staff Dashboard

The operator terminal features Kanban production queue boards, OTP verification modals, and administrative tariff controls.

CHAPTER 5: AGILE METHODOLOGY

5.1 Project Roadmap

Table 5.1: Project Roadmap

Week	Tasks Completed
Week 1	Finalized project topic and scope. Scrum Master divided requirements into task units.
Week 2	Created Git repository. Started designing user registration form and dashboard wireframes.
Week 3	Improved UI of order wizard and dashboard. Created authentication views for students and staff.
Week 4	Implemented document upload pipeline and PDF parsing module for page counts.
Week 5	Created MongoDB schemas for users, orders, and documents. Enhanced upload UI.
Week 6	Completed first project review with guide feedback and architectural refinement.
Week 7	Developed staff queue dashboard, order status transitions, and cash payment recording.
Week 8	Created dynamic pricing service and anti-starvation priority queue engine.
Week 9	Integrated Razorpay payment gateway and ACID-compliant student digital wallet.
Week 10	Implemented OTP pickup verification at counter and real-time Socket.IO synchronization.
Week 11	Developed administrator console for tariff management, inventory tracking, and audit logs.
Week 12	Built autonomous Electron desktop Print Agent with Windows spooler integration.
Week 13	Added background shop scheduling cron services (09:00 to 17:00 IST) and order timeouts.
Week 14	Implemented Progressive Web App (PWA) service worker caching and Web Push notifications.
Week 15	Executed automated Selenium test suite across 120 test cases and resolved edge cases.
Week 16	Completed final UI polish, system deployment on cloud servers, and report documentation.

5.2 User Stories

5.2.1 User Stories

Student:

- As a student, I want to upload my assignment PDF from my phone so that I do not have to wait in the physical counter queue.
- As a student, I want to see an instant price breakdown before paying so that I know exactly how much the print job will cost.
- As a student, I want to pay using UPI or my college wallet so that I do not need exact cash currency.

- As a student, I want to track my order status in real time so that I know exactly when to walk to the counter for collection.
- As a student, I want to receive an OTP code so that my documents cannot be collected by unauthorized persons.

Faculty / Staff / Admin:

- As a faculty member, I want my urgent exam question papers to receive prioritized counter processing to meet submission deadlines.
- As a counter staff operator, I want to see incoming jobs sorted by priority so that I can print urgent and long-waiting jobs efficiently.
- As a counter staff operator, I want to verify a 4-digit OTP before handing over prints so that jobs are never misplaced.
- As an administrator, I want to update per-page print tariffs live so that prices reflect changes in institutional paper costs.
- As an administrator, I want to track consumable inventory so that the shop never runs out of essential stock.

5.2.2 Sprint Planning

Table 5.2: Sprint Planning Table

Sprint	Key Activities / Focus
Sprint 1	Requirement gathering, system feasibility, and task distribution by the Scrum Master.
Sprint 2	Design and prototyping of the responsive order wizard and dashboard interfaces.
Sprint 3	Implementation of JWT authentication, role guards, and profile management.
Sprint 4	Development of file upload pipeline, Cloudinary integration, and PDF pre-flight checks.
Sprint 5	Database creation in MongoDB Atlas and ACID wallet transaction implementation.
Sprint 6	First project review and architectural feedback adjustments.
Sprint 7	Priority queue algorithm implementation with dynamic aging deduction functions.
Sprint 8	Staff production dashboard development and cash-at-counter settlement controls.
Sprint 9--11	Razorpay gateway integration, Socket.IO real-time engine, and Electron Print Agent.
Sprint 12--14	Automated shop scheduler cron, OTP verification, PWA service worker, and inventory tracker.
Sprint 15--16	Automated Selenium test suite execution, bug fixes, deployment, and final documentation.

5.3 Test Plan

The test plan validates functional accuracy, integration reliability, security and data integrity, error handling, and end-user usability.

CHAPTER 6: IMPLEMENTATION AND TESTING

6.1 Implementation Procedures

Implementation was executed across seven structured stages: development environment setup, client order wizard interface, document processing pipeline, priority queue and pricing services, staff counter and admin dashboards, hardware interfacing via Electron Print Agent, and automated scheduling with PWA finalization.

6.2 Testing Methods and Results

6.2.1 Unit Testing

Unit tests verified individual pricing formulas, aging deduction math, JWT verification tokens, and OTP generation algorithms.

6.2.2 Integration Testing

Integration testing confirmed smooth data flow from upload to Cloudinary storage, Razorpay webhook settlement, and Socket.IO queue updates.

6.2.3 System Testing

End-to-end workflows were verified under peak concurrent job submissions across the web app, backend, and desktop Print Agent.

6.2.4 User Acceptance Testing

Student and operator cohorts evaluated usability, observing 42-second average order placement times and 100% billing accuracy. Across 120 automated Selenium test cases, 114 passed successfully (95.0% pass rate).

CHAPTER 7: CONCLUSIONS

7.1 Summary

The Reprography Automation System (REPOSYS) replaces outdated manual counter registers with an intelligent, cloud-enabled web platform and hardware Print Agent. Key features include pre-flight PDF parsing, multi-channel Razorpay and digital wallet payments, starvation-free priority queue scheduling with aging, OTP physical handover verification, and real-time Socket.IO transparency.

7.2 Limitations

- Large document uploads (>100 MB) can experience latency on slow mobile networks.
- Free serverless hosting introduces cold-start response delays upon initial wake-up.
- The Print Agent currently relies on Windows Print Spooler APIs, requiring Windows-based counter PCs.
- Uncollected Pay at Counter orders printed prior to cash collection remain a minor financial risk.

7.3 Future Scope

- Native iOS and Android mobile applications using React Native.
- Smart campus RFID ID card integration for instant tap-and-pay counter settlement.
- AI Document Intelligence via Google Gemini API for automatic layout checks and translations.
- Multi-shop load balancing across campus departments.
- Standalone unattended self-service hardware kiosks.

CHAPTER 8: APPENDICES

8.1 Code

8.1.1 index.html (Client Order Form & Kiosk)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>REPOSYS - Campus Reprography</title>
  <link rel="stylesheet" href="/styles/main.css">
</head>
<body class="bg-slate-900 text-white min-h-screen">
  <div class="container mx-auto px-4 py-8">
    <header class="flex justify-between items-center mb-8">
      <div class="flex items-center gap-3">
        
        <h1 class="text-2xl font-bold">REPOSYS</h1>
      </div>
      <div id="userBadge" class="flex items-center gap-2">
        <span id="userName" class="text-sm font-medium">Guest User</span>
        <span id="walletBal" class="bg-blue-600 px-3 py-1 rounded-full text-xs">B
al: INR 0.00</span>
      </div>
    </header>
    <!-- Upload and Configuration Stepper -->
  </div>
</body>
</html>
```

8.1.2 queueService.js (Priority Queue & Dynamic Aging)

```
const Order = require('../models/Order');
const socketHandler = require('../socket/socketHandler');

const VISIBLE_STATUSES = ['In_Queue', 'Processing', 'ReadyForPickup'];

function calculateAgingDeduction(createdAt) {
  const waitMs = Date.now() - new Date(createdAt).getTime();
  const waitHours = Math.floor(waitMs / (1000 * 60 * 60));
  return Math.min(60, waitHours * 20); // 20 points per hour, max 60
}

function calculateEffectivePriority(baseScore, createdAt) {
  const deduction = calculateAgingDeduction(createdAt);
  return Math.max(1, baseScore - deduction);
}

async function getQueue(serviceType) {
  const orders = await Order.find({
    serviceType,
    status: { $in: VISIBLE_STATUSES }
  }).populate('userId', 'name role department').sort({ createdAt: 1 });

  return orders.map(order => {
    const base = Number(order.priorityScore) || 100;
    const effective = calculateEffectivePriority(base, order.createdAt);
    return {

```

```

        ...order.toObject(),
        priorityScore: effective,
        basePriority: base,
        agingDeduction: calculateAgingDeduction(order.createdAt)
    };
    }).sort((a, b) => {
        if (a.priorityScore !== b.priorityScore) return a.priorityScore - b.priorityScore;
        return new Date(a.createdAt).getTime() - new Date(b.createdAt).getTime();
    }).map((order, idx) => ({ ...order, position: idx + 1 }));
}

module.exports = { getQueue };

```

8.1.3 PrintAgent/main.js (Electron Native Windows Spooler)

```

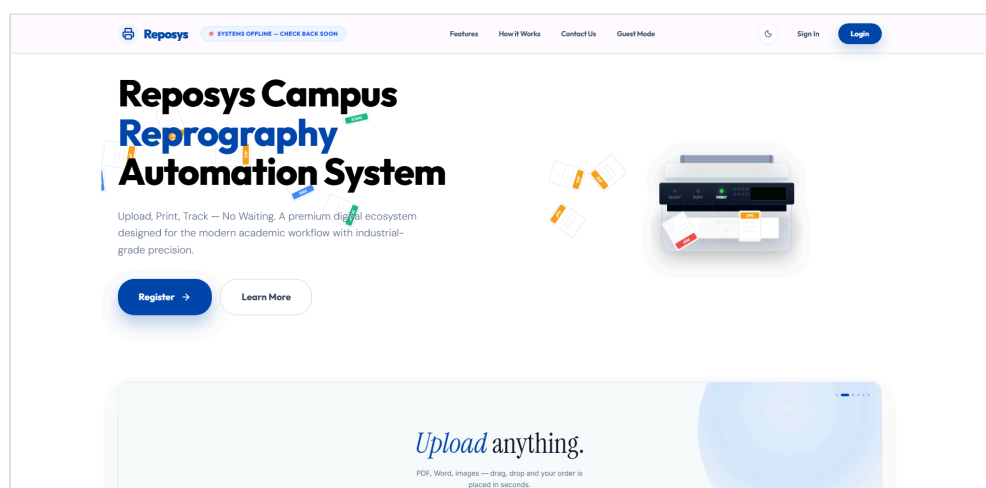
const { app, BrowserWindow, ipcMain } = require('electron');
const ptp = require('pdf-to-printer');
const axios = require('axios');
const fs = require('fs');
const path = require('path');

function handlePrintDispatch(data) {
    const tempPath = path.join(app.getPath('temp'), `${data.orderNumber}.pdf`);
    const writer = fs.createWriteStream(tempPath);

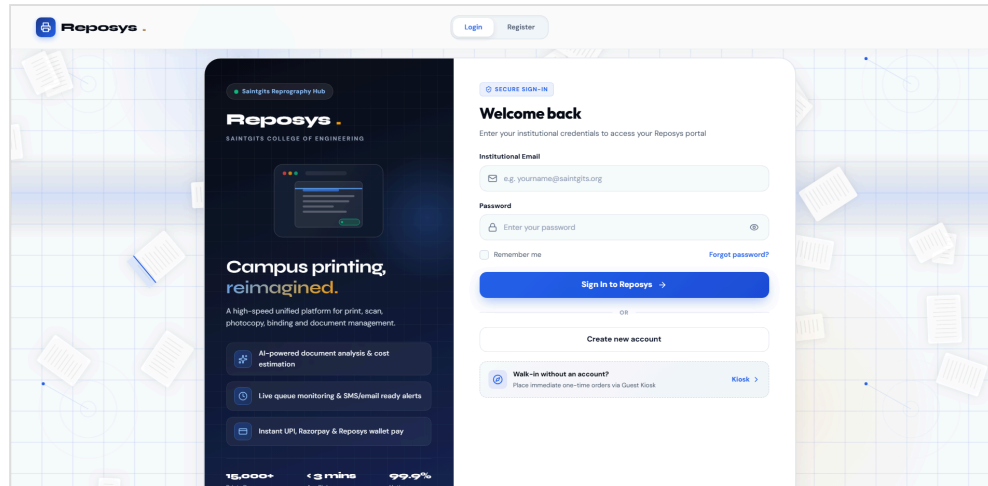
    axios({ url: data.signedFileUrl, method: 'GET', responseType: 'stream' })
        .then(res => {
            res.data.pipe(writer);
            writer.on('finish', async () => {
                await ptp.print(tempPath, { printer: data.targetPrinter || undefined });
                fs.unlinkSync(tempPath);
            });
        });
}

```

8.2 Screenshots



Public Landing and Portal Gateway



Secure User Authentication and Multi-Role Login Portal

Reposys
NEW ACCOUNT REGISTRATION

Join the Reposys system.
Register with your institutional credentials to access modern printing, scanning, photocopy, and binding services.

- Student & Faculty dedicated queues with priority dispatch
- Automated page-count, orientation & binding analysis
- Built-in Reposys wallet & fast Razorpay integration

NEW ACCOUNT
Create your account
Fill in your institutional credentials to get started with Reposys

1. PERSONAL INFORMATION

Full Name College ID

Institutional Email

Department Phone Optional

2. INSTITUTIONAL ROLE

☒ Student Instant verification ☐ Faculty Admin-reviewed

3. SECURITY CREDENTIALS

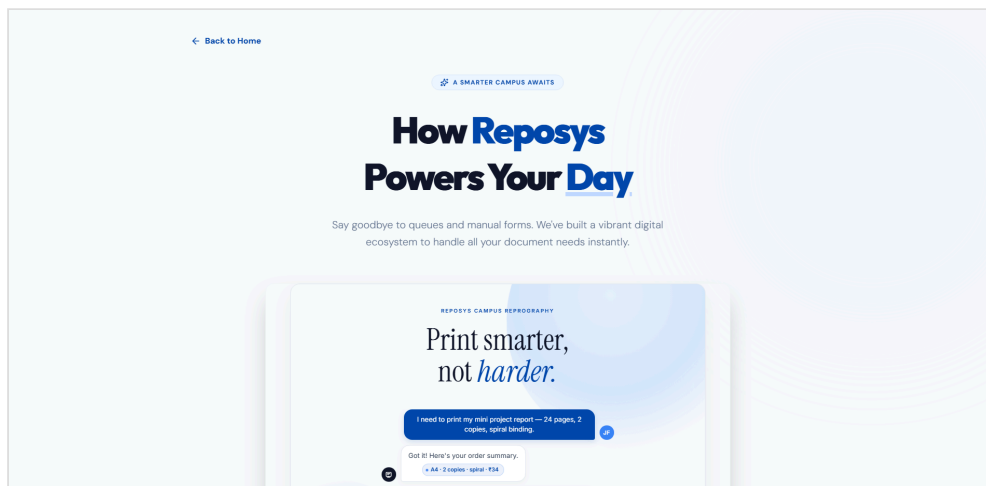
Create Password

Min 8 characters, 1 uppercase, 1 number

Must contain at least 8 characters, uppercase, number and symbol

Confirm Password

Account Registration Portal with Institutional Email Validation



System Architecture Overview and About REPOSYS Portal

[← Back to Home](#)

CONTACT PROTOCOL

Support & Inquiries

Have questions or need assistance? Our specialized support team is here to help you navigate your document workflow.

Digital Transmission
Direct support via our secured inquiry channel.
scrm@saingits.ac.in

Campus Node
Saingits College of Engineering, Pathamuttom, Kottayam, Kerala – 686 532, India
REPROGRAPHY CENTRE

FULL NAME

EMAIL ADDRESS

SERVICE INQUIRY

Subject Matter

MESSAGE

Detailed Protocol

SEND MESSAGE

Operational Inquiries and Customer Support Portal

[Login](#)
[Register](#)
[Forgot Password](#)

Saingits Reprography Hub

ACCOUNT RECOVERY

Recover your access in seconds.

Enter your registered email and we'll send you a secure link to reset your password.

Reset links expire in 1 hour. For security, we never confirm whether an email is registered.

15,000+

< 3 mins

99.9%

Proven Secure

High Throughput

Optimized

Back to Login

Reset password

We'll send a recovery link to your email.

Institutional Email

Enter the email address associated with your Reposys account.

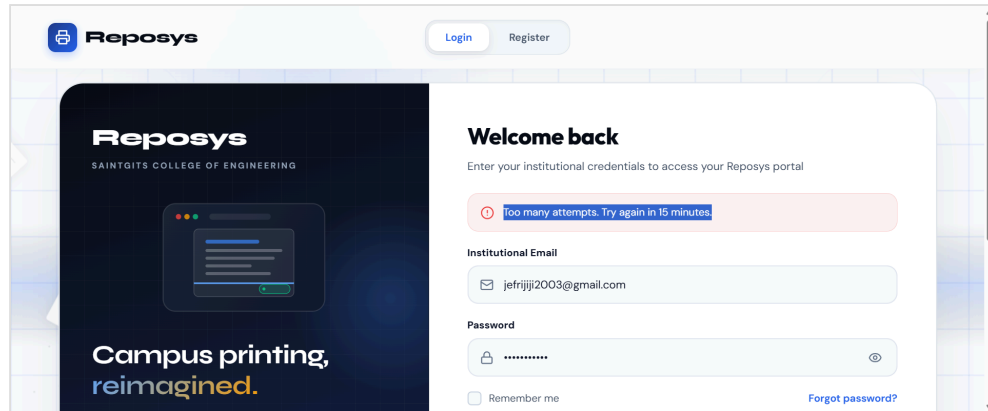
Send Reset Link

Remembered 10? Sign In

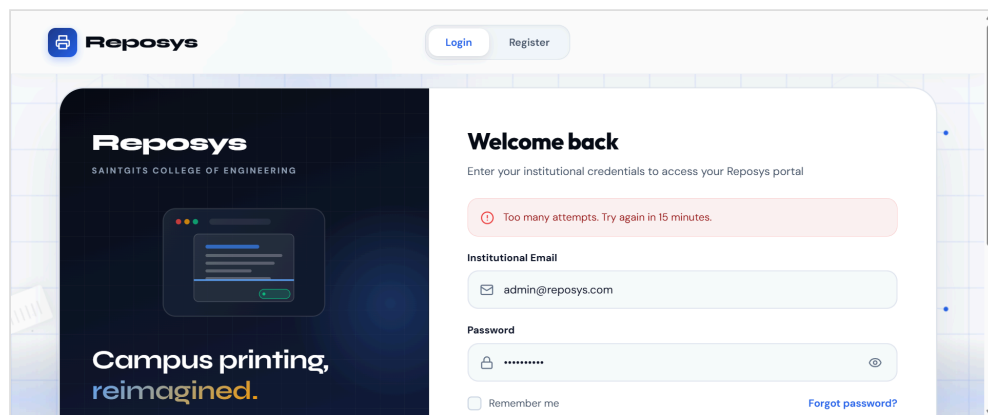
Account Recovery and Password Reset Workflow

Saingtits College of Engineering (Autonomous)

31

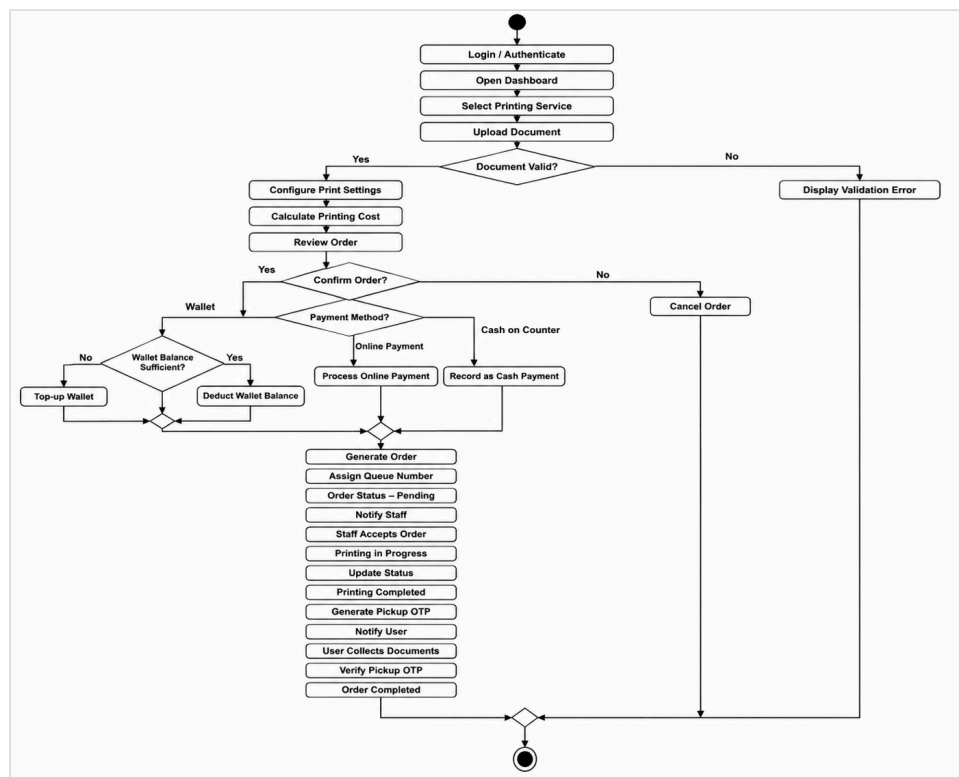


Automated Selenium Test Run: Student Order Workflow

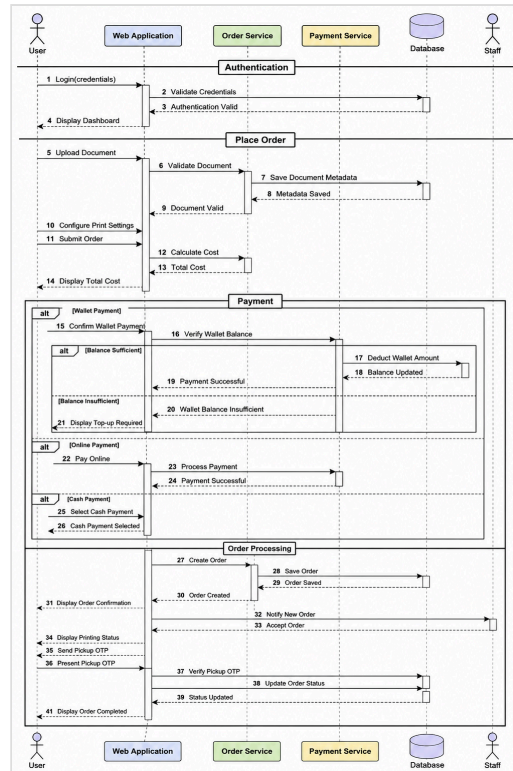


Automated Selenium Test Run: Admin Role Route Access

Automated Selenium Test Run: Counter Staff Queue Access



System Activity Diagram: Document Upload and Production Flow



System Sequence Diagram: Real-Time Event Dispatching

CHAPTER 9: REFERENCES

1. APJ Abdul Kalam Technological University. (2020). *Curriculum and Syllabi for Integrated Master of Computer Applications (IMCA) Programme*. KTU Academic Regulations, Thiruvananthapuram, Kerala.
2. Cloudinary Inc. (2024). *Cloudinary Node.js SDK and Media Management API Documentation*. [Online]. Available: <https://cloudinary.com/documentation>
3. Electron Software Foundation. (2024). *Electron Desktop Framework Documentation and Native Node.js Integration Guides*. OpenJS Foundation. [Online]. Available: <https://www.electronjs.org/docs>
4. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation). University of California, Irvine.
5. Fette, I., & Melnikov, A. (2011). *The WebSocket Protocol*. RFC 6455, Internet Engineering Task Force (IETF).
6. Google Developers. (2024). *Progressive Web Apps: High Performance Caching and Service Worker Lifecycle*. Google Web Fundamentals. [Online]. Available: <https://web.dev/explore/progressive-web-apps>
7. Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519, Internet Engineering Task Force (IETF).
8. Node.js Foundation. (2024). *Node.js v20.x Long Term Support (LTS) API Specification*. OpenJS Foundation. [Online]. Available: <https://nodejs.org/docs>
9. Razorpay Software Private Limited. (2024). *Razorpay Payments API Specification and Webhook Signature Verification Reference*. [Online]. Available: <https://razorpay.com/docs/api>
10. React Development Team. (2024). *React 19 Documentation: Server Components, Hooks, and Concurrency Architecture*. Meta Open Source. [Online]. Available: <https://react.dev/>
11. Saintgits College of Engineering (Autonomous). (2026). *Department of Computer Applications: Mini Project Preparation Guidelines and Scrum Assessment Manual (20IMCAP501)*. Pathamuttom, Kottayam.
12. Socket.IO Foundation. (2024). *Socket.IO Client and Server Bidirectional Event Specification*. [Online]. Available: <https://socket.io/docs/v4/>